Assignment 2: Deep Q-Network

MLDS Advanced Algorithms for AI: CartPole-v1 and MsPacman-v5

Yihao Chen

November 11, 2026

Submission information

Item	Value
GitHub username	`1haochen`
Repository	`mlds-reinforcement-learning-assignment2`
Public link	https://github.com/1haochen/mlds-reinforcement-learning-assignment2
Hardware	deepdish3, **one** NVIDIA RTX 2080 Ti per MsPacman run (`CUDA_VISIBLE_DEVICES`), CPU for CartPole
Software	Python 3.10, PyTorch 2.5.1, Gymnasium 1.2.0, ale-py 0.11.2

How to run the code

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
# T1-T3 analytic & pipeline tests (~15 s, CPU)
.venv/bin/python validate.py analytic
# all MsPacman + CartPole runs (one GPU per MsPacman run), then the CartPole sweep A-F
GPU_MAIN=0 GPU_ABL=1 TIME=9000 bash run_all.sh
bash sweep_cartpole.sh
# 500-episode greedy evaluation on held-out seeds 100000-100499
.venv/bin/python dqn.py evaluate --checkpoint results/cartpole_s1/best.pt --episodes 500
.venv/bin/python dqn.py evaluate --checkpoint results/mspacman/best.pt --episodes 500 \
    --workers 20
# behavioural tests T4-T8
.venv/bin/python validate.py cartpole --checkpoint results/cartpole_s1/best.pt
.venv/bin/python validate.py mspacman --checkpoint results/mspacman/best.pt
.venv/bin/python validate.py failure_modes --checkpoint results/cartpole_s{0,1,2}/best.pt \
    results/cartpole_s{0,1,2}/last.pt
# figures, then this report
.venv/bin/python plots.py --cartpole results/cartpole_s1 --mspacman results/mspacman
python3 build_report.py && pandoc report_full.md -o Assignment2_Report.pdf --pdf-engine=xelatex
```

Each test's purpose and expected outcome is listed in `README.md` and in Section 5.

1. Implementation (`dqn.py`)

Algorithm. Standard DQN with experience replay and a target network, plus Double-DQN targets. For a transition (s, a, r, s', d) the loss is the Huber loss between $Q_\theta(s, a)$ and

$$y = r + \gamma\,(1 - d_{\text{terminated}})\; Q_{\bar{\theta}}\big(s', \arg\max_{a'} Q_{\theta}(s', a')\big),$$

where $\bar{\theta}$ is the **target network**. It is a hard copy of θ, refreshed every C gradient updates. Gradients are clipped to norm 10 and optimised with Adam.

Component	CartPole-v1	MsPacman-v5
Network	MLP 4-128-128-2 (ReLU)	Nature CNN on 4x88x80: conv 32@8x8/4, 64@4x4/2, 64@3x3/1, FC 512, 9 outputs
State	raw 4-d observation	`preprocess_observation` (assignment function) with the last 4 frames stacked
Discount γ	0.95	0.99
Replay buffer	50k transitions, uniform; 1k warm-up	200k transitions, uniform, 20k warm-up; s' stored as one frame and rebuilt from s
Target network	hard copy every 500 updates	hard copy every 2,500 updates (= 10k env steps)
Loss / optimiser	Huber, Adam lr $10^{-3} \to 10^{-4}$ (linear over 100k steps)	Huber, Adam lr $10^{-4} \to 2 \cdot 10^{-5}$ (linear LR scheduler over 6M steps; the 2.5 h budget ended at 3.1M steps, lr $\approx 5.8 \cdot 10^{-5}$)
Exploration	ε: 1 → 0.05 over 10k steps, then → 0.01	ε: 1 → 0.1 over 1M steps, then → 0.01 over 2M steps
Batch / replay ratio	64, 1 update per env step	32, 2 updates per 8 env steps (8 parallel envs)
Reward used for learning	+1 per step	sign-clipped to $\{-1, 0, +1\}$; raw game score is what we plot

Engineering choices. (i) *Vectorised actors.* MsPacman uses 8 `AsyncVectorEnv` workers that feed one learner on one GPU. The replay ratio stays the Nature value (one 32-sample update per 4 env steps). (ii) *Memory.* The buffer stores s (4 frames) and only the newest frame of s', which cuts memory by about 45%. `validate.py` T3 checks that s' is rebuilt exactly. (iii) *Checkpoint selection.* Every k episodes the greedy policy plays validation seeds 10000+ and we keep `best.pt`. The final 500-episode evaluation uses **disjoint** seeds 100000–100499, so the selection is not scored on its own data. (iv) *Logging.* Every episode we log reward, length, ε, learning rate, loss, the mean and max of $\max_a Q(s_t, a)$ along the episode, and the action counts.

2. Edge cases and how they are handled

1. **Termination vs. truncation (time limit).** CartPole-v1 stops at 500 steps with `truncated=True`, and MsPacman-v5 has a frame cap. A truncated state is *not* terminal, so the target must still bootstrap. We store only `terminated` as the done-mask. T2 checks the target values directly, and the CartPole calibration plot (Fig. 5) shows Q staying near 20 right up to step 500.
2. **Autoreset in vectorised envs.** After an episode ends, Gymnasium's default autoreset returns the *reset* observation. Storing that as s' would silently corrupt every terminal transition. We use `AutoresetMode.SAME_STEP` and take s' from `info["final_obs"]`. T3 checks on the real game that the stored frame equals the true final frame and differs from the reset frame.
3. **Frame stack at episode start.** A new episode's stack is filled with 4 copies of its first frame, so frames never leak from the previous episode (T3).
4. **uint8 overflow in preprocessing.** The RGB sum can reach 765. `ndarray.sum` promotes to a wider integer, so it does not wrap. We store the frame as uint8 `img//3`, which is the assignment's int8 value + 128 and therefore lossless; the network maps it back to $[-1, 1)$. T3 checks that a white pixel gives 255 and a Ms. Pac-Man pixel gives 0.

5. **Replay buffer wrap-around and warm-up.** Insertion uses a ring index. Sampling only starts once the buffer holds `warmup ≥ batch` transitions (enforced by the argument parser), so we never sample from empty slots.
6. **Unbounded / large rewards.** MsPacman rewards range from 10 to 1,600 per step. They are sign-clipped for learning, which bounds $|Q| \leq 1/(1-\gamma) = 100$, and the raw scores are reported.
7. **Loss of lives in MsPacman.** Losing a life is not treated as terminal. The env's `terminated` flag (game over) is the only terminal signal, so the agent optimises the real objective.
8. **Short-horizon discounting in CartPole ($\gamma = 0.95$).** The effective horizon is $1/(1-\gamma) = 20$ steps. A slow drift of the cart toward $|x| = 2.4$ that ends the episode 150 steps later changes Q by only $20 \cdot 0.95^{150} \approx 0.009$. We observed exactly this failure mode (Section 5, T8). It is the main reason CartPole needs a small learning rate, a large buffer and checkpoint selection.
9. **Greedy loops / ties.** Evaluation is greedy ($\varepsilon = 0$). MsPacman-v5 keeps its sticky actions (p = 0.25), so a deterministic policy cannot loop forever, and the frame cap bounds episode length in any case.
10. **Single GPU.** Each MsPacman process sees exactly one card via `CUDA_VISIBLE_DEVICES`.

3. Part (a): CartPole-v1 ($\gamma = 0.95$)

Final model. Main configuration from `run_all.sh`, seed 1 (`results/cartpole_s1`): lr $10^{-3} \rightarrow 10^{-4}$, batch 64, target copy every 500 updates, 600 episodes. We kept the checkpoint with the best greedy validation score (episode 560, 10/10 validation episodes at 500). We confirmed it on a second, independent validation set (seeds 200000+, 50/50 episodes at 500) before touching the test seeds.

(iii) 500 roll-outs (test seeds 100000–100499, greedy): mean = 497.34, standard deviation = 34.27, median = 500; 497/500 episodes (99.4%) reach the 500-step limit. The environment is solved.

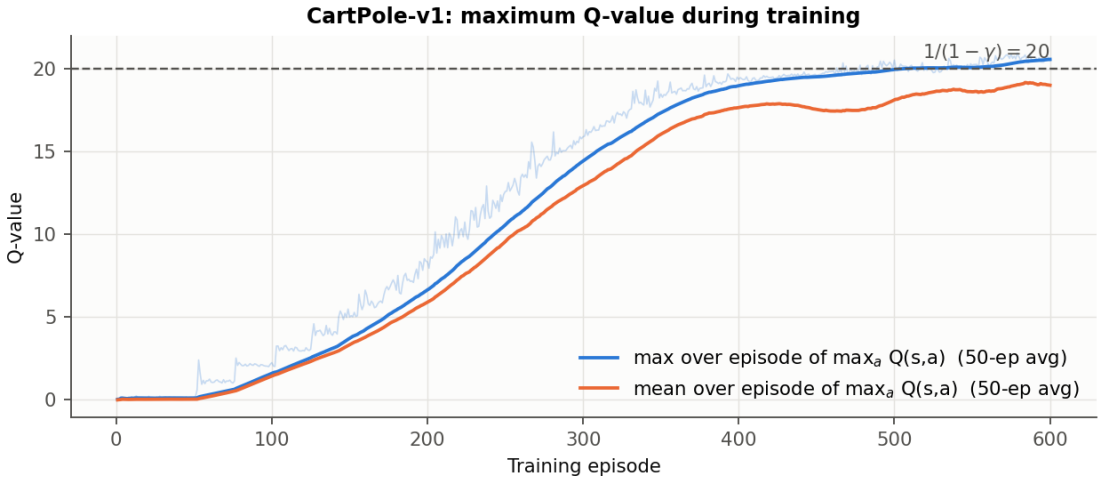


Figure 1: CartPole: maximum Q-value during training. Thin line = per-episode maximum; thick lines = 50-episode averages.

(i) Q-values. Every step pays +1, and a never-failing policy only ever stops by time-limit truncation, which is *not* terminal. So the optimal value is $Q^* = \sum_k \gamma^k = 1/(1-0.95) = 20$. The learned maximum Q rises smoothly from 0 and **levels off right at 20** (dashed line). At start states of the final model its mean is **20.01**. Overshooting 20 would indicate over-estimation; levelling off well below 20 would mean truncation was being treated as terminal.

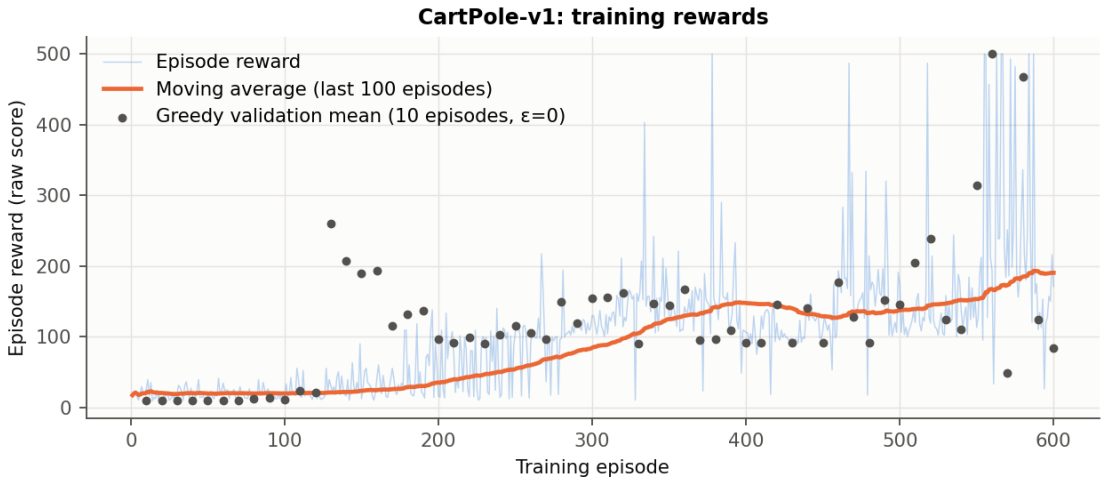


Figure 2: CartPole: training rewards (ε-greedy) with 100-episode moving average and greedy validation means.

(ii) Rewards. The 100-episode moving average climbs from about 20 (random play) to about 190 by episode 600. The greedy validation points (black) show what the policy can do without exploration noise: they reach 500 at episode 560 and 467 at episode 580. Training rewards understate the policy. Even at $\varepsilon = 0.01$ one random action can start a fall, and the T4 noise test shows that $\varepsilon = 0.05$ already costs about 19% of the return.



Figure 3: CartPole: histogram of 500 greedy evaluation episodes (log-scale counts).

Stability across seeds and hyper-parameters. DQN on CartPole is well known to oscillate after first solving it, and $\gamma = 0.95$ makes this worse (edge case 8). To report this honestly rather than show one lucky run, we trained **27 runs**: the main configuration, two ablations and six sweep configurations, each with 3 seeds. The table counts how often the greedy validation return hit 500.

Configuration	seeds with a 500 checkpoint	first 500 (episode, per seed)	validation checks at 500	best val. (per seed)	mean val., 2nd half
main (lr 1e-3, tgt 500)	1/3	–, 560, –	1	288, 500, 454	139
no Double DQN	2/3	520, 340, –	3	500, 500, 219	157

Configuration	seeds with a 500 checkpoint	first 500 (episode, per seed)	validation checks at 500	best val. (per seed)	mean val., 2nd half
no target network	0/3	–, –, –	0	88, 9, 9	9
A: lr 5e-4, batch 128, tgt 1000	1/3	–, –, 1000	1	462, 333, 500	150
B: lr 2.5e-4, batch 128, tgt 2000	1/3	260, –, –	1	500, 386, 474	138
C: lr 1e-3, 2 updates/step	3/3	830, 460, 710	23	500, 500, 500	259
D: lr 3e-4, batch 256, 1500 ep	3/3	1470, 940, 660	18	500, 500, 500	227
E: lr 1e-4 const, tgt 250, 1500 ep	2/3	1020, 160, –	5	500, 500, 479	201
F: E + dueling head	2/3	140, 1390, –	6	500, 500, 472	204

Every configuration with a target network produced a 500-validation checkpoint in at least one seed, while **none** of the 9 no-target checkpoints exceeded 88. Configurations that do more or larger gradient updates per sample were the most reliable: C (2 updates per step) and D (batch 256) solved in **3/3 seeds**, and C hit 500 at 23 validation checks. That fits the action-gap explanation in edge case 8: Q must be learned very precisely before the tiny difference between "push left" and "push right" in safe states rises above the noise. No configuration stays at 500 permanently ("mean val., 2nd half" is 138–259), and the dueling head (F) did not fix this. Checkpoint selection on validation seeds is what turns an oscillating learner into a reliable policy.

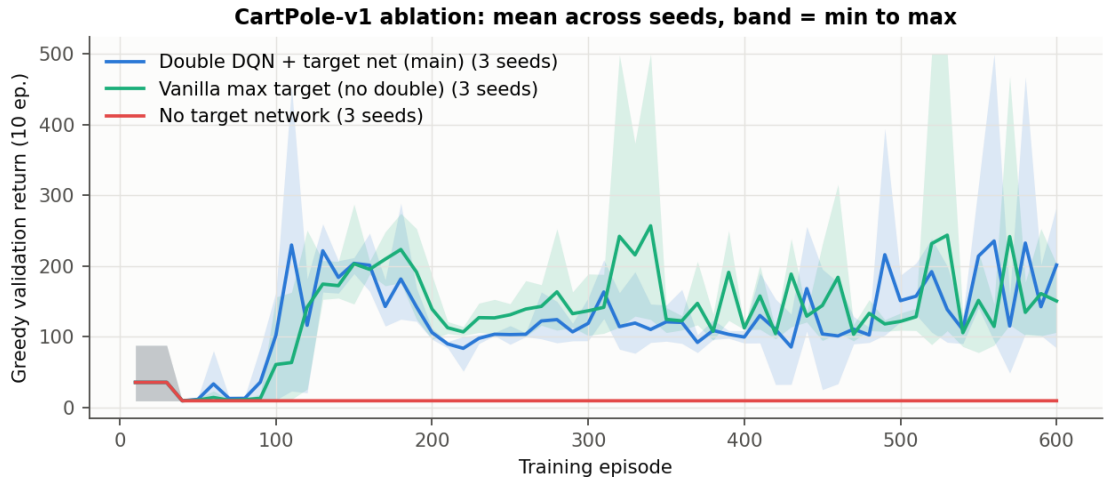


Figure 4: CartPole ablation: greedy validation return of the main configuration vs. no Double-DQN vs. no target network (3 seeds each; band = min to max).

Non-standard plot: value calibration. Below we compare the predicted $\max_a Q(s_t, a)$ with the return the agent *actually* collected afterwards in a held-out episode. For 450 steps the two agree at about 20, so the network has learned the true discounted value. In the last 50 steps the realised return collapses only because the 500-step time limit cuts the episode short. Q correctly does *not* fall: that state is not terminal, the policy could keep balancing, and we never mask truncations. Had we treated truncation as terminal, Q would have learned to predict this artificial drop from the state alone, which it cannot see, so the target would be noisy.

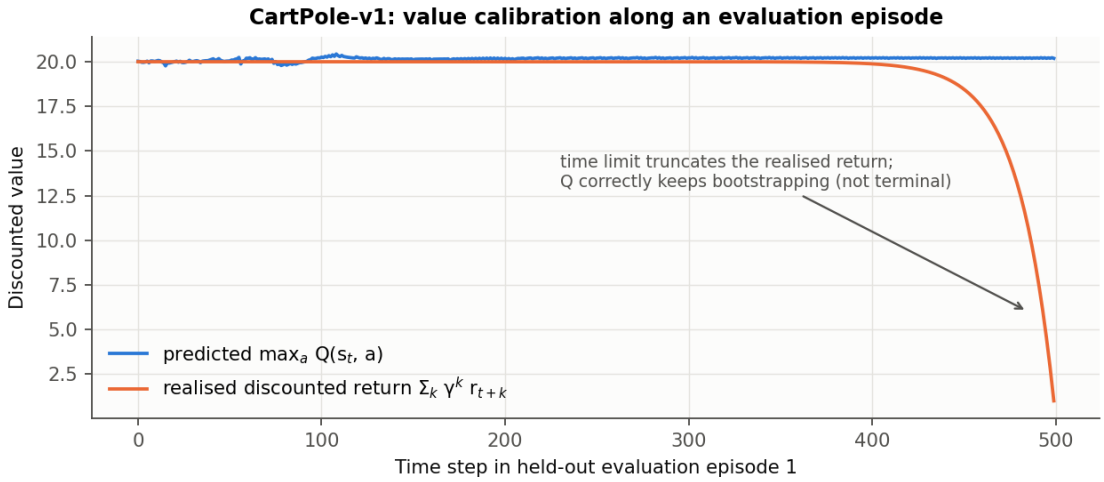


Figure 5: CartPole: predicted value vs. realised discounted return along a held-out evaluation episode.

First 10 steps of a held-out episode (`results/cartpole_s1/evaluation/first10.json`):

t	x	$\dot{x}$	θ	$\dot{\theta}$	Q(left)	Q(right)	action
1	−0.037	−0.003	0.050	0.032	19.99	20.03	right
2	−0.038	0.191	0.050	−0.245	19.98	20.01	right
3	−0.034	0.386	0.045	−0.521	19.95	19.97	right
4	−0.026	0.580	0.035	−0.799	19.91	19.96	right
5	−0.014	0.775	0.019	−1.080	19.90	19.99	right
6	0.001	0.970	−0.002	−1.367	20.01	20.04	right
7	0.021	1.165	−0.030	−1.661	19.94	**12.16**	**left**
8	0.044	0.970	−0.063	−1.377	20.05	15.60	left
9	0.063	0.776	−0.091	−1.105	20.04	18.38	left
10	0.079	0.582	−0.113	−0.842	20.02	19.56	left

The pole starts leaning right ($\theta > 0$), so the agent pushes right to get under it. Once the pole has swung past vertical and is falling left fast ($\dot{\theta} = -1.66$), another push right would be fatal. Q(right) drops to 12.2, about 8 points below Q(left), and the agent switches. While the state is safe the two actions differ by only about 0.05, the tiny action gap discussed in edge case 8.

4. Part (b): MsPacman-v5 ($\gamma = 0.99$)

Training run (`results/mspacman`, one RTX 2080 Ti): **4,614 episodes**, 3.13M agent steps (12.5M emulator frames at frame-skip 4), 776k gradient updates, and a 2.5-hour wall-clock budget set by the deadline. The run stopped on that budget, not because learning had levelled off. The checkpoint with the best greedy validation score (episode 4,300, mean 1,720 over 5 validation seeds) was evaluated on 500 held-out seeds.

(iii) 500 roll-outs (test seeds 100000–100499, greedy, sticky actions on): mean = 1,664.8, standard deviation = 497.1, median 1,570, min 900, max 4,570. For reference, a uniformly random policy scores 233 ± 54 on the same game (T4, $\varepsilon = 1$). Even the trained agent's *worst* test episode (900) is far above that.

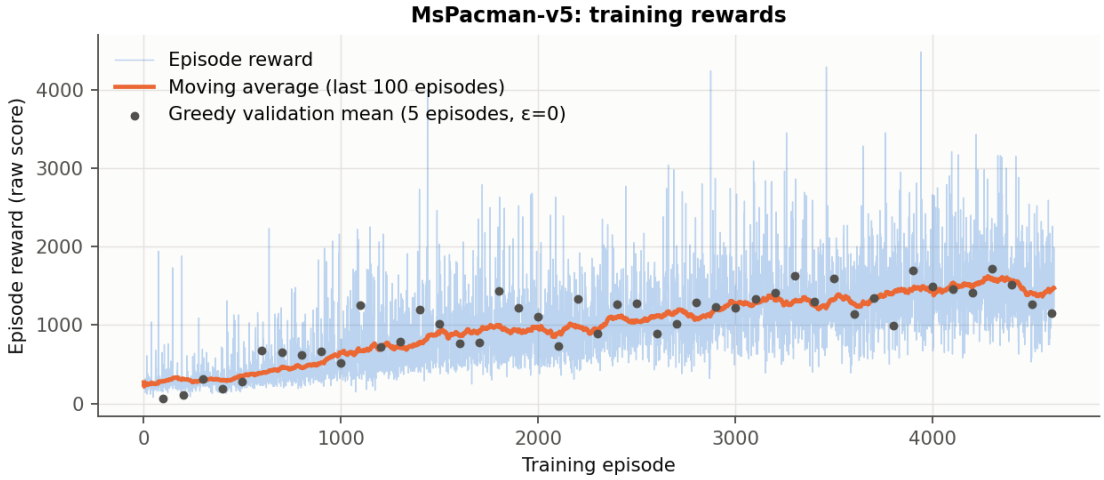


Figure 6: MsPacman: training rewards with 100-episode moving average and greedy validation means.

(ii) Rewards: clear learning. The 100-episode moving average rises from **282** (first 100 episodes, essentially random play) to **1,473** (last 100), peaking near 1,600 around episode 4,300. The trend holds at every stage of the ε schedule. At first the gains come from the policy itself while ε is still above 0.5 (episodes 500–1,000; $\varepsilon = 0.5$ at episode 1,013). They continue after ε reaches its floor (0.1 at about episode 1,700, then 0.01). The greedy validation means (black dots, 5 episodes each, so noisy) track the moving average.

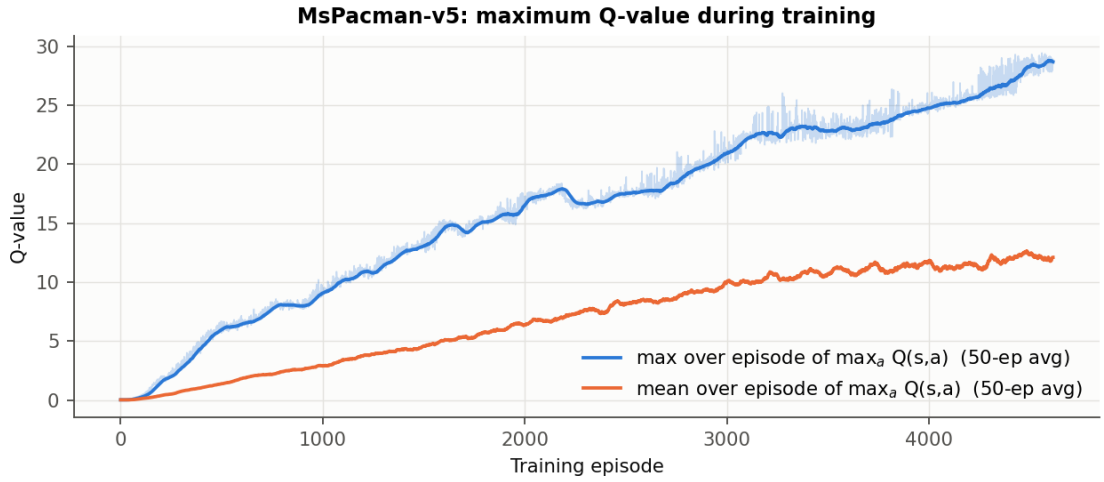


Figure 7: MsPacman: maximum Q-value during training.

(i) Q-values. Q rises smoothly and monotonically as the agent learns to collect more reward per life, from 0 to a per-episode maximum of about 29 and an episode mean of about 12. There are no spikes and no plateau at an implausible level. With rewards clipped to $\{-1, 0, 1\}$ and $\gamma = 0.99$, $|Q| \leq 100$ always holds. 29 is consistent with eating about one pellet every two steps for a while ($\sum_k 0.99^{2k} \approx 50$), discounted by the chance of dying. The curve had not levelled off when the time budget ran out, consistent with the reward curve.

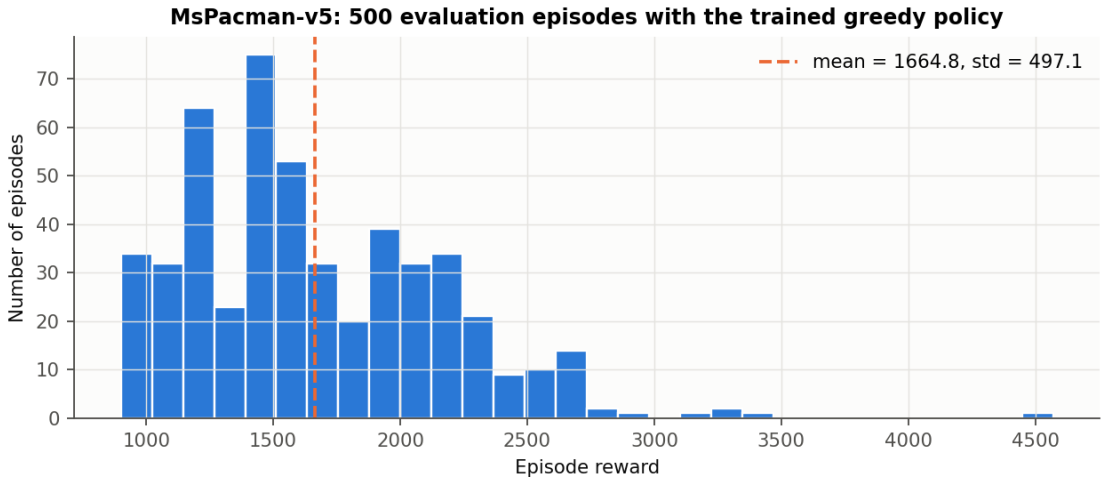


Figure 8: MsPacman: histogram of 500 greedy evaluation episodes.

The distribution is clearly multi-modal, with peaks near 1,200, 1,450 and 2,000. In Ms. Pac-Man a run's score is dominated by *discrete* events: whether the agent clears the first maze (all pellets) and how many ghosts it eats after a power pellet (200/400/800/1,600). So scores cluster by how many such events happened rather than spreading as a bell curve.

Non-standard plot: value calibration on a held-out episode. Through most of the game, predicted $\max_a Q$ tracks the realised discounted (clipped) return closely. Over the 20 calibration episodes the correlation is 0.76. The visible failure is instructive: during the **frozen screens** at the start of the game and after each lost life (flat blue segments at steps 0–65, about 460–490 and about 580–610), Q falls to 2–4 while the true value is 5–25. On those screens nothing moves and the 4-frame stack holds 4 identical frames, so the network cannot "see" how far the freeze has progressed. It also receives zero reward for about 65 steps, so the value must be bootstrapped through a long chain of indistinguishable states. Section 7 lists a targeted fix.

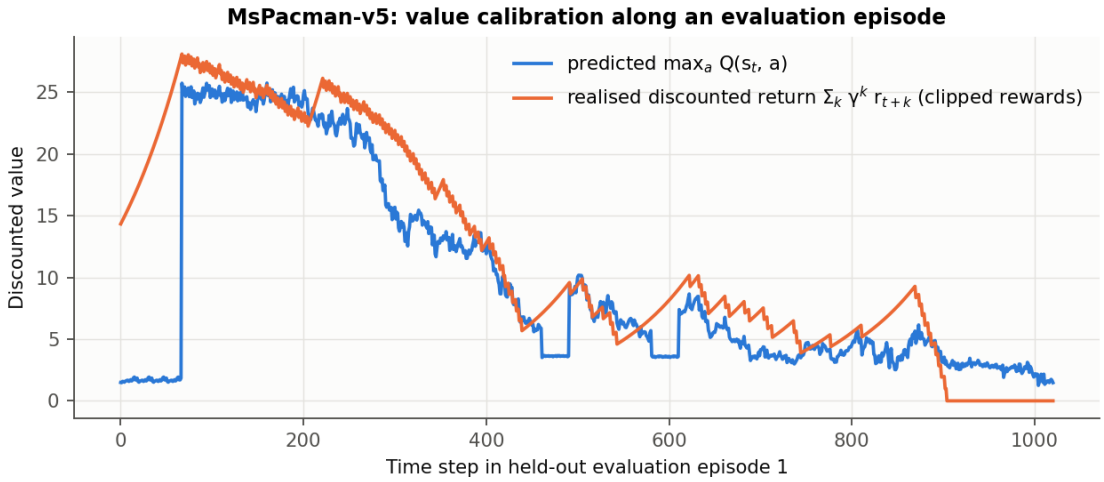


Figure 9: MsPacman: predicted value vs. realised discounted return along a held-out evaluation episode.

Target-network ablation: score vs. correctness of values. We trained an identical second agent *without* a target network (bootstrapping from the online network), on a second GPU, for the same 2.5 hours (Fig. 10). Unlike CartPole, it did **not** collapse. It learned more slowly at first, then overtook the main run late and scored **2,008.8 ± 499.5** on the same 500 test seeds, against 1,664.8 ± 497.1. The value estimates, however, show the problem the target network is meant to prevent:

	main (target net + Double DQN)	no target network
test score (500 ep.)	1,664.8 ± 497.1	2,008.8 ± 499.5
mean predicted Q vs. realised return (20 eval. ep.)	10.9 vs. 12.4 (**under**-estimates)	15.5 vs. 10.7 (**over**-estimates by 45%)
largest Q seen during training	29.4	**228.4**, above the hard bound of 100

Right after learning starts, the no-target network **diverged**: Q exceeded 220 for 8 straight episodes, impossible for clipped rewards. It recovered only because the learning rate is small and gradients are clipped. It then remained about 45% optimistic and spiky for the rest of training. The target network kept Q bounded, smooth and slightly pessimistic, the known effect of Double DQN. **With one seed per arm we cannot claim either agent scores better.** CartPole (3/3 no-target seeds collapsed) and the bound violation here both show that the target network is what keeps the regression target stable. Running more seeds of this comparison is the first item in Section 7.

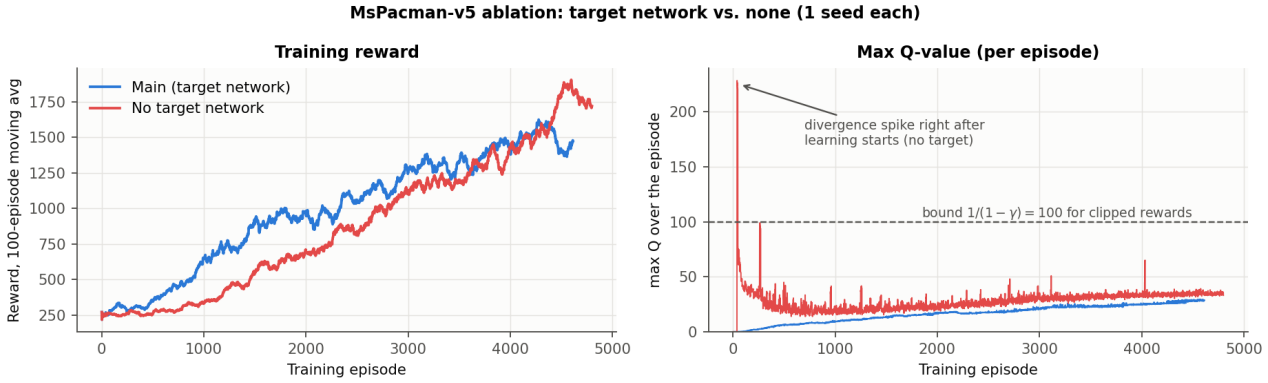


Figure 10: MsPacman ablation: training reward (left) and per-episode max Q (right) with and without a target network.

5. Tests for validating correctness

Each test changes something whose effect we can **predict in advance**, then checks that the implementation behaves as predicted. All of them run from `validate.py` (commands in the header) and write JSON to `results/validation/`.

T1. Analytic chain MDP: does the DQN machinery converge to the right numbers? (`validate.py analytic`) A 4-state corridor with one-hot states: "right" from the last state pays +1 and ends the episode, and every other move pays 0. The closed form is $Q^*(s, \text{right}) = \gamma^{3-s}$ and $Q^*(s, \text{left}) = \gamma\, V^*(\max(s-1, 0))$. We train it with the **exact** `ReplayBuffer`, `dqn_update` (Double-DQN + Huber + clipping) and target-network code used for the games.

γ	max |learned − closed form|	greedy = optimal?
0.00	3.0e-08	yes
0.50	1.2e-07	yes
0.95	2.0e-07	yes
0.99	6.4e-06	yes
0.95, **terminal flag ignored** (deliberate bug)	**14.71**	values drift toward $1/(1-\gamma) = 20$

The learned values match the analytic Q* to within 1e-3 for every γ. The bug variant shows the test is sensitive: drop the $(1-d)$ mask and the same code learns values above 12 where the truth is at most 1.

T2. Terminal vs. truncated targets. Feed `td_target` the same $r = 1$ and $V(s') = 20$ with $\gamma = 0.95$. It gives **1.0** when terminated and **20.0** when truncated, matching the expected values exactly. The CartPole Q level of 20.01 (Fig. 1) and the calibration plot (Fig. 5) show the same behaviour end-to-end.

T3. Pipeline edge cases on the real MsPacman environment (all pass):

- the Ms. Pac-Man colour (sum 448) is blacked out, and a white pixel (sum 765) maps to 255 with no uint8 wrap-around;
- a reset stack is 4 copies of the first frame, both at the start and for a new episode inside the vector env;
- at the first real game-over (step 478 of a random roll-out) the stored s' frame equals `info["final_obs"]`, differs from the reset frame, and has `terminated = 1`;
- s' rebuilt from (s, new frame) in the replay buffer is bit-identical to the true next stack for every transition.

T4. Policy-noise intervention (`validate.py cartpole` / `mspacman`, 100 and 30 paired seeds). *Prediction:* if the greedy action really carries the learned value, return falls monotonically as we replace it with random actions, and $\varepsilon = 1$ matches a random policy.

ε at test time	0	0.05	0.1	0.25	0.5	1.0
CartPole mean reward	500.0	406.8	303.5	97.5	26.5	22.0
CartPole fraction at 500	1.00	0.80	0.58	0.12	0.00	0.00

ε at test time	0.0	0.05	0.25	1.0
MsPacman mean score (30 seeds)	1525	1520	1188	233
MsPacman mean episode length	873	845	705	490

Both games behave as predicted: return never increases as random actions are injected, and $\varepsilon = 1$ reproduces random play (CartPole about 22 steps; MsPacman 233, against 1,525 for the greedy agent). The *shape* differs between the games, which is informative. CartPole collapses fast, because a single wrong push at the wrong moment is unrecoverable: $\varepsilon = 0.05$ already costs 19%. MsPacman is almost unaffected at $\varepsilon = 0.05$ (−0.3%) and loses only 22% at $\varepsilon = 0.25$, because one wrong joystick move in a maze is usually harmless and corrected on the next step. (The $\varepsilon = 0$ MsPacman value of 1,525 on 30 seeds is within sampling error of the 500-episode mean, 1,665 ± 497.)

T5. Changing the physics (CartPole, 30 episodes per setting, Fig. 11). *Prediction:* the policy implicitly encodes the dynamics, so it should be perfect at the training values and degrade as constants move away. "Easier" changes (lighter pole, weaker gravity, stronger motor) should hurt less than "harder" ones. Measured: 500/500 at the training physics. Halving gravity (4.9), a lighter pole (0.05) and a stronger motor (force 15 or 20) are all still perfect. A weaker motor (force 5) collapses to 20 steps, a 5x heavier pole (0.5) drops to 421, and doubled gravity to 100. Changing the pole *length* in either direction hurts, because the policy's timing is tuned to that pendulum period. This matches the prediction and shows the agent is a control law tuned to the training dynamics, not a lookup of memorised trajectories.

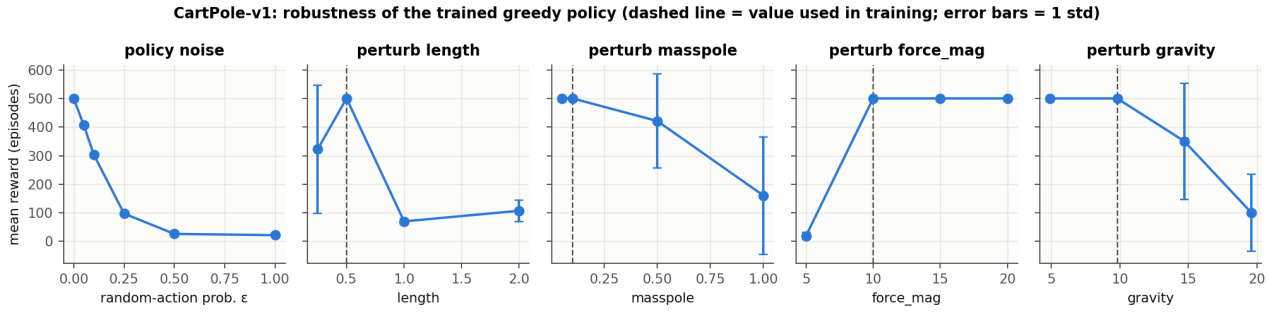


Figure 11: CartPole: policy noise (left) and physics perturbations (right four).

T6. Mirror symmetry (a negative finding). CartPole's dynamics are symmetric under $s \to -s$ with left↔right, so we expected a correct Q to flip its greedy action on mirrored states. Measured flip rate: **0.57** on confident states, barely above chance. The cause is the data, not the code. The final policy parks the cart left of centre (mean $x = -0.91$; **95% of visited**

states have $x < 0$), so mirrored states ($x \approx +0.9$) are almost never in the replay buffer. DQN only learns Q where it has data. This also explains the x-boundary failures in T8. A cheap fix would be to augment the buffer with mirrored transitions (Section 7).

T7. Q-level vs. theory. CartPole: the learned $\max_a Q$ at start states is **20.01** against the theoretical 20.00 (0.05% error). MsPacman: with clipped rewards, $|Q| \leq 1/(1 - 0.99) = 100$. The learned values stay far inside that bound for the main agent (largest Q seen in training: 29.4) and track the realised return (mean 10.9 predicted vs. 12.4 realised). The no-target agent broke the bound (228.4), a clear sign of divergence (Section 4).

T8. Failure-mode diagnosis (CartPole). We replayed 20 held-out episodes from the best and the final (episode 600) checkpoints of the three main-config seeds and recorded *why* each episode ended:

checkpoint (main config)	mean length	ended by pole angle	ended by |x| > 2.4	truncated at 500
best of seed 0 (ep 180)	298	20	0	0
best of seed 2 (ep 110)	415	19	0	1
best of seed 1 (ep 560, final model)	500	0	0	20
last of seed 0 (ep 600)	224	4	**16**	0
last of seed 1 (ep 600)	91	1	**19**	0
last of seed 2 (ep 600)	286	1	**16**	3

(`validate.py failure_modes`, 20 held-out episodes each, output in `results/validation/failure_modes.json`.)

Early checkpoints fail because the pole falls. By episode 600 the agent has learned to balance, and 51 of the 60 episodes end because the cart *drifts off the track*. That is precisely the long-horizon failure predicted by $\gamma = 0.95$: the cost of drifting is discounted below the noise in Q (edge case 8). It is why the learning curves oscillate and why we select checkpoints by validation.

T9. Ablations (Figs. 4 and 10). Removing the target network makes CartPole learning collapse: all 3 seeds end with greedy return of about 9, the minimum possible, because Q chases its own moving target and diverges. On MsPacman the no-target agent did not collapse, but it diverged right after learning started (Q above 220, past the hard bound of 100) and stayed about 45% over-optimistic. Its final score was higher (2,008.8 vs. 1,664.8) in this single-seed comparison, so the score difference is not conclusive (Section 4).

6. Plot that is non-standard

Besides the Q-calibration plots (Figs. 5 and 9), the robustness panel (Fig. 11) and the Q-value ablation (Fig. 10), the figure below shows **how the behaviour policy's action distribution evolves** during MsPacman training. Each column is a block of episodes, and colour is the share of steps on which each of the 9 joystick actions was chosen. The top axis shows ε. While $\varepsilon \approx 1$ all actions are equally likely (uniform colour). As ε falls, the greedy preferences take over. The shifts are moderate, because many joystick actions are near-equivalent in a maze (e.g. UPRIGHT acts like UP or RIGHT depending on the corridor). From an almost uniform 11% each in episodes 0–500, the policy moves to prefer UP (13.0%) and RIGHT (12.9%) and to avoid NOOP (8.9%) and LEFT (9.4%) by episodes 1,500–2,500. In the last 300 episodes DOWNRIGHT (13.6%) and UPRIGHT (12.5%) become the most used. Two checks follow from the plot. NOOP is never useful for scoring, and the agent learns to use it least. No row ever becomes solid dark, which would point to a degenerate policy that always picks one action (e.g. an argmax over an untrained or diverged network).

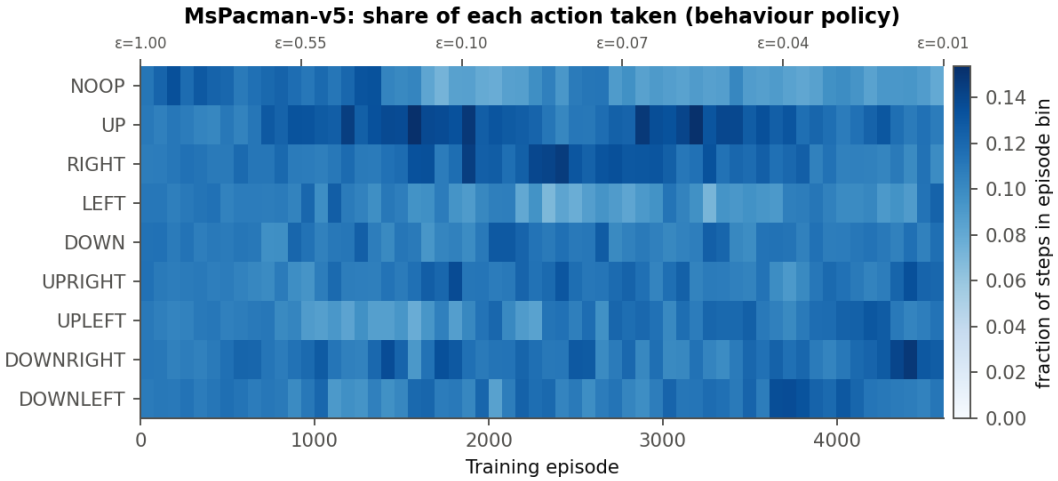


Figure 12: MsPacman: evolution of the action distribution over training.

7. With 3 more months: further validation tests and experiments

1. **Target network vs. none on MsPacman with 5+ seeds.** Our single-seed result (the no-target agent scored higher but diverged and over-estimated) needs seeds before any score claim. We would also try soft (Polyak) target updates as a middle ground.
2. **Fix and test the frozen-screen value error (Fig. 9).** Add a "frames since the last change" feature or a longer frame stack, and test the prediction that Q on frozen start/respawn screens then matches the realised return.
3. **Throughput.** Training was limited by CPU-side emulation (GPU about 13% busy), not by the GPU. Options: 32–48 environment workers or EnvPool, a GPU-resident replay buffer, and larger batches with mixed precision. That would allow the 10–50M-frame runs below on the same single card.
4. **Statistical rigour.** 10+ seeds per configuration, with bootstrap confidence intervals (e.g. interquartile mean, as in the `rliable` protocol) instead of single-seed curves, and a permutation test for "Double-DQN vs. vanilla".
5. **Over-estimation audit.** For both games, measure the gap between predicted Q and Monte-Carlo return over the whole visited-state distribution for vanilla vs. Double DQN, and how it grows with the number of actions. Compare against the analytical bias of a max over noisy estimates.
6. **Mirror-augmented replay for CartPole.** Add $(-s, 1-a, r, -s')$ for each transition and test whether T6's flip rate goes to 1 and the x-drift failures (T8) disappear.
7. **Horizon test.** Train CartPole with $\gamma \in \{0.9, 0.95, 0.99, 0.995\}$ and check whether the share of x-boundary failures falls as $1/(1-\gamma)$ grows, a direct test of the edge-case-8 explanation.
8. **Longer MsPacman training (10–50M frames)** with Rainbow components (prioritised replay, n-step returns, dueling, distributional C51, noisy nets), one at a time, to measure each contribution.
9. **Generalisation tests for MsPacman.** Other mazes/levels, no sticky actions vs. sticky 0.25, and different `repeat_action_probability` and `frameskip` values. This separates memorised routes from a reactive policy.
10. **Interpretability.** Saliency or Grad-CAM maps over the frame stack to check that Q attends to ghosts, pellets and Ms. Pac-Man rather than to the score digits. A **controlled-state test**: place a ghost next to Ms. Pac-Man in a saved emulator state and check that Q of the action moving toward it drops.
11. **Replay diagnostics.** Distribution of sample age and TD-error over training; buffer-size sweep to check whether catastrophic forgetting (early failures pushed out of the buffer) drives the CartPole oscillations.
12. **Hyper-parameter search** (Bayesian optimisation / successive halving) over learning rate, target period, replay ratio and ε schedule, with checkpoint selection always done on validation seeds disjoint from test seeds.